

Praktikum Pemrograman Berorientasi Objek

Pertemuan 3

Materi Pertemuan 3

1. Inheritance

2. this & super keywords

3. Overriding

4. Overloading

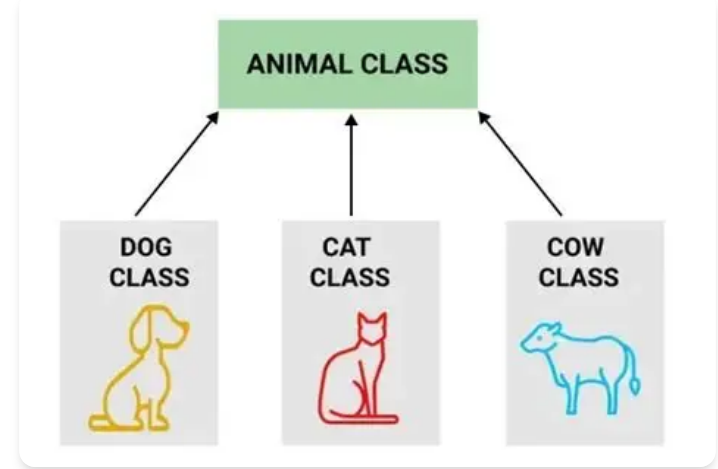
Inheritance

Mewarisi karakteristik dari sebuah class

Definisi Inheritance

Secara bahasa, inheritance adalah pewarisan. Inheritance adalah mekanisme untuk mewarisi karakteristik dari sebuah class. Dari sisi penulis kode, inheritance dapat digunakan untuk mengembangkan fitur di class baru (child class) dari class asalnya (parent class). Dan biasanya (meski tidak selalu), terdapat hubungan yang semantik antara parent dan child class.

Merupakan salah satu konsep di OOP yang menyatakan bahwa suatu class dapat memiliki class turunan (sub-class). Secara umum, konsep inheritance ini digunakan untuk menerima properti atau karakteristik dari class lain yang memiliki hubungan is-a.



Mengapa konsep inheritance muncul?

Konsep inheritance pertama kali muncul pada tahun 1960-an melalui bahasa pemrograman Simula 67, yang dirancang untuk memodelkan fenomena objek di dunia nyata. Di tengah kompleksitas pemodelan tersebut, inheritance hadir sebagai solusi untuk mereduksi duplikasi kode dan memfasilitasi klasifikasi taksonomi.

Namun, hal ini memunculkan miskonsepsi: inheritance sering dianggap murni sebagai fasilitas desain di mana setiap objek wajib memiliki hubungan representatif (hubungan is-a), padahal hal tersebut tidak sepenuhnya benar.

Dalam Java, inheritance ditandai dengan penggunaan kata kunci `extends`. Kata kunci ini digunakan untuk menyatakan bahwa suatu class merupakan turunan dari class lain.

```
public class Mobil extends Kendaraan {  
  
}
```

Pada contoh di atas, `extends` dipakai untuk menunjukkan bahwa class `Mobil` mewarisi seluruh karakteristik (atribut dan method) dari class `Kendaraan`.

Kapan Inheritance digunakan?

1. Kebutuhan untuk modifikasi incremental, pada definisi dijelaskan jika konsep ini memungkinkan untuk mengembangkan fitur di class baru, adaptasi ini memungkinkan modifikasi dengan membuat definisi baru dan menjadi bagian sistem lama tidak rusak.
2. Terdapat struktur taksonomi (is-a relationship) pada sistem yang dikembangkan, kasus paling umum di mana inheritance digunakan ketika sebuah class memiliki hubungan representatif dengan class lain.
3. Kebutuhan untuk code reusability, meski tidak memiliki hubungan konseptual yang nyata, inheritance dapat digunakan untuk code reuseability jika sebuah class memiliki karakteristik yang mirip dengan class lain.

Tujuannya apa sih:

- DRY (Don't Repeat Yourself)
- Code Reusability

Dengan adanya konsep ini lebih memudahkan programmer jika menghadapi situasi dimana ada 2 class berbeda yang memiliki behaviour yang sama sehingga method tidak perlu ditulis ulang dengan catatan kedua class tersebut memiliki hubungan is-a.

Catatan mengenai inheritance

Java hanya dapat mewarisi dari satu class saja, berbeda dengan C++ yang memungkinkan melakukan pewarisan dari dua atau lebih class. Namun, multi inheritance menimbulkan masalah baru, yaitu diamond problem.

Batasan Inheritance:

- **Diamond Problem** -> permasalahan yang terjadi ketika child-class memiliki lebih dari 1 parent-class (multiple inheritance) dimana child-class meng-inherit properties dan methods dari 2 parent class yang memiliki method dengan nama dan parameter yang sama. Karena ada diamond problem ini, maka java tidak mengizinkan adanya Multiple Inheritance satu child-class di java hanya boleh memiliki 1 parent-class.
- **Harus memperhatikan access modifier** dengan teliti sebelum menurunkan suatu class. Sebagai contoh jika di parent-class memiliki atribut dengan access modifier private, maka atribut tersebut tidak dapat diakses secara langsung.

Contoh penerapan inheritance

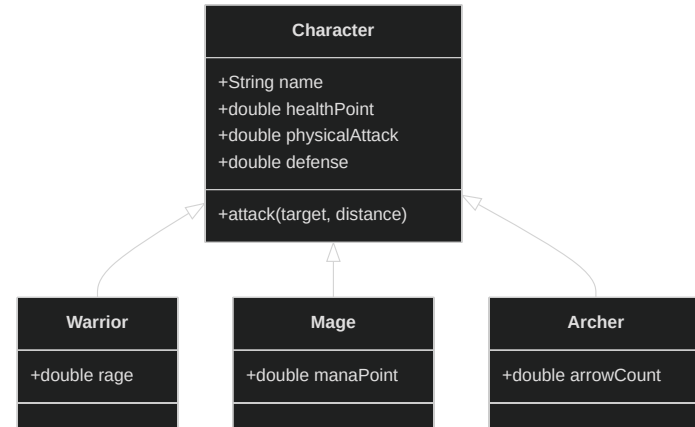
(studi kasus is-a relationship / general to specialization)

Pada sistem game ini, terdapat class `Character` sebagai sistem dasar (*parent class*) untuk turunan karakter lainnya.

Sistem game tersebut memiliki tiga spesialisasi (*child class*):

- `Warrior` (Serangan jarak dekat)
- `Mage` (Serangan area sihir)
- `Archer` (Serangan jarak jauh)

Setiap spesialisasi mewarisi seluruh atribut dan method dasar dari `Character` (seperti `name` , `healthPoint` , dll), namun bisa memiliki atribut tambahan yang spesifik sesuai spesialisasinya (contoh: `rage` untuk `Warrior`).



Implementasi: Parent Class (Character)

Pertama, kita definisikan class `Character` sebagai fondasi. Class ini berisi atribut dan method dasar yang merepresentasikan logika umum sebuah karakter.

```
public class Character {
    protected String name;
    protected double healthPoint;
    protected double physicalAttack;
    protected double defense;

    // Method umum yang akan diwariskan ke seluruh child class
    public void attack(Character target, double distance) {
        if (distance > 1.5) {
            System.out.printf("%s terlalu jauh untuk diserang!\n", target.name);
            return;
        }

        double damage = Math.max(physicalAttack - target.defense, 0);
        target.healthPoint -= damage;
        System.out.printf("%s menerima kerusakan dasar sebesar %.0f\n", target.name, damage);
    }
}
```

Implementasi: Child Class (Warrior, Mage, Archer)

Dengan keyword `extends` , class turunan secara otomatis memiliki atribut (seperti `name` , `healthPoint`) dan method `attack()` dari class `Character`.

```
public class Warrior extends Character {  
    // Atribut tambahan khusus Warrior (jika ada)  
    private double rage;  
}
```

```
public class Mage extends Character {  
    // Atribut tambahan khusus Mage  
    private double manaPoint;  
}
```

```
public class Archer extends Character {  
    // Atribut tambahan khusus Archer  
    private double arrowCount;  
}  
  
// Saat dieksekusi di Main:  
// Warrior w = new Warrior();  
// w.attack(mageTarget, 1.0);  
// → otomatis memanggil method attack() dari parent!
```

Access modifier di inheritance

Access modifier tidak terbatas pada public dan private saja, **protected** kadangkala digunakan di inheritance, tujuannya agar class turunannya dapat mengakses langsung atribut dan/atau method dari class parentnya tanpa memerlukan getter dan setter.

Meski praktik ini, dianggap melanggar encapsulation.

Access Modifier	Class yang Sama	Package yang Sama	Subclass	Class Manapun
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

this & super keywords

Mereferensikan objek dari class itu sendiri dan parent class

Kata kunci this & super

- `this` Keywords -> tugasnya adalah menunjuk atribut (variable) atau method milik class itu sendiri
- `super` Keywords -> tugasnya adalah menunjuk atribut (variable) atau method milik parent-classnya

Ketika menulis kode di child class, seringkali kita menggunakan `super` . `super` adalah kata kunci yang merepresentasikan objek dari parent class.

`super` sama seperti `this` atau nama variabel yang didefinisikan sebagai objek dari sebuah class. `this` merepresentasikan objek di class itu sendiri, `super` merepresentasikan objek untuk parent classnya.

Contoh this & super (Studi Kasus Game)

```
public class Character {
    protected String name;
    protected int currentLevel;

    public Character(String name, int currentLevel) {
        // 'this' membedakan atribut milik class dengan parameter
        this.name = name;
        this.currentLevel = currentLevel;
    }
}

public class Warrior extends Character {
    private double healthPoint;
    public Warrior(String name, int currentLevel) {
        // 'super' memanggil dan mengirim data ke constructor parent
        super(name, currentLevel);
    }

    // Constructor Warrior yang memiliki parameter lebih lengkap
    public Warrior(String name, int currentLevel, double healthPoint/) {
        super(name, currentLevel);
        this.healthPoint = healthPoint;
    }
}
```

Method Overriding

Mendefinisikan ulang implementasi dari method

Method Overriding

Definisi

Method overriding lebih kayak nulis ulang method yang udah ada sehingga memungkinkan subclass untuk menyediakan implementasi spesifik dari sebuah metode yang sudah didefinisikan di kelas induknya (parent class).

Pada contoh sebelumnya, dikatakan bahwa setiap karakter memiliki karakteristik serangan yang berbeda. Karakteristik yang berbeda dari class utama ini dapat diimplementasikan dengan konsep method overriding.

Mekanismenya adalah mendefinisikan ulang method dengan nama, return type, dan parameter yang sama, tetapi penerapan di dalamnya berbeda. **Wajib tambahkan** `@Override` agar Java tahu ada method yang ditimpa.

Access modifier

Access modifier di sub-classnya tidak boleh lebih ketat dari parent class.

Contoh Method Overriding (Studi Kasus Game)

```
// --- Di class Character ---
public void attack(Character target, double distance) {
    // Logika serangan dasar karakter
}

// --- Di class Warrior ---
@Override
public void attack(Character target, double distance) {
    // Logika khusus Warrior: jarak serang harus pendek
    if (distance < minRange || distance > maxRange) {
        System.out.printf("%s terlalu jauh untuk diserang!\n", target.name);
        return;
    }

    double damage = Math.max(physicalAttack - target.defense, 0);
    target.takeDamage(damage);
    System.out.printf("%s menerima kerusakan fisik sebesar %.0f\n",
        target.name, damage);
}
```

Method Overloading

Satu nama method dengan variasi parameter

Method Overloading

Method overloading adalah mekanisme di mana kita bisa menulis ulang method dengan nama yang sama, tetapi dengan return type, tipe data parameter, urutan parameter, atau jumlah parameter yang berbeda serta implementasi method yang berbeda juga.

Compiler akan secara otomatis menentukan fungsi mana yang paling cocok dengan inputan yang diberikan.

Syarat:

- List parameter harus **berbeda** (jumlah, tipe data, urutan).
- **TIDAK BOLEH** overload hanya berdasarkan perbedaan return type (untuk overloading method).

Implementasi method overloading pada attack()

Implementasi awal

```
@Override
public void attack(Character target, double distance) {
    // Logika damage normal...
}
```

Implementasi overloading

```
public void attack(Character target, double distance, boolean isPowerStrike) {
    if (!isPowerStrike) {
        this.attack(target, distance);
        return;
    }

    if (distance < minRange || distance > maxRange) {
        System.out.printf("%s terlalu jauh untuk diserang!\n", target.name);
        return;
    }

    double damage = Math.max((physicalAttack * 1.5) - target.defense, 0);
    target.takeDamage(damage);
    System.out.printf("%s menerima kerusakan skill fisik sebesar %.0f\n", target.name, damage);
}
```

Constructor Overloading

Overloading juga bisa diterapkan pada constructor. Konsep ini berarti membuat banyak constructor yang memiliki jumlah atau tipe parameter yang berbeda-beda. Pada sistem `Character` kita, terdapat tiga versi *constructor* sekaligus. Kita bisa membuat objek karakter kosongan, membuat dengan nama saja, atau membuat dengan spesifikasi yang sangat lengkap.

```
// 1. Default constructor
public Character() {

}

// 2. Parameterized constructor (1)
public Character(String name, int currentLevel) {
    this.name = name;
    this.currentLevel = currentLevel;
}

// 3. Parameterized constructor (2)
public Character(String name, int currentLevel, double healthPoint, // ... dan parameter lainnya) {
    this.name = name;
    this.currentLevel = currentLevel;
    this.healthPoint = healthPoint;
    // ... dan atribut lainnya
}
```

Latihan Minggu 3

Download boilerplate di link berikut:

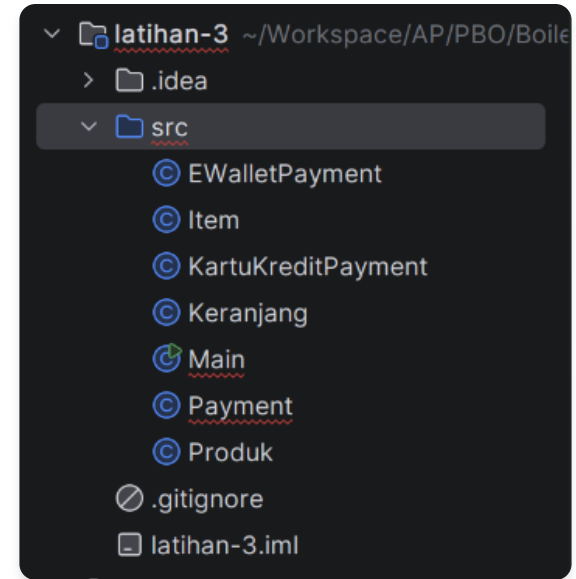
https://drive.google.com/drive/folders/1T92VVBdKWpGb6x5Udslt4lvz_Edw0ZOl?usp=sharing

Instruksi Latihan

Misal diketahui ingin diimplementasi sistem checkout dari sebuah toko online. Terdapat beberapa opsi pembayaran, secara tunai, kartu kredit, dan e-wallet.

Setiap opsi bayar direpresentasikan dengan classnya masing-masing, seperti Payment, KartuKreditPayment, dan EWalletPayment. Implementasikan konsep yang sudah dipelajari untuk mengisi flow kode yang belum lengkap.

```
// TODO: Buat class Payment agar memiliki dua atribut jumlahDibayar  
// Gunakan access modifier yang sesuai  
public class Payment { 2 usages  
  
    // TODO: Buat constructor kosong  
  
    // TODO: Buat method prosesPembayaran  
    // 1. Menerima satu parameter, amount dengan tipe double  
    // 2. Set properti jumlahDibayar dengan penambahan PPN 11%  
    // 3. Panggil method informasiPembayaran  
    // 4. Set properti waktuPembayaran dengan LocalDateTime.now()
```



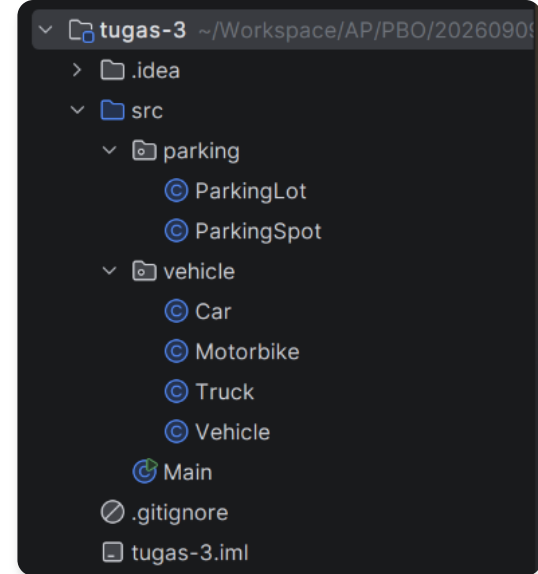
Assignments

Silahkan cek Classmoji masing-masing, tugas dapat diakses setelah dipublish dan diinformasikan oleh asprak kelas masing-masing.

Instruksi Assignment

Silahkan clone repositori terkait, tidak perlu menulis kode dari nol. Implementasikan konsep-konsep yang sudah dipelajari untuk melengkapi kode-kode yang belum lengkap. Kerjakan hanya bagian kode yang bertuliskan TODO.

```
// TODO: Buat method checkIn  
// 1. Buat method yang menerima objek dari class Car, Motorbike, atau Truck  
// 2. Cari spot kosong, dengan method di class ini, jika spot tersedia  
// parkirkan kendaraan  
// Hint: Untuk parameter class Car, Motorbike, atau Truck bisa gunakan Vehicle  
public void checkIn() { 6 usages  🧑 Haris Herdiansyah  
  
}
```



Teknis Pengumpulan

Teknis pengumpulan tugas akan diinformasikan dikemudian hari.

Deadline Pengumpulan

Kelas A:

14 September 2026

Kelas B:

13 September 2026

Kelas C:

15 September 2026

Waktu yang dilihat adalah waktu last commit. Jika ada yang commit melewati deadline walaupun sudah commit sebelumnya akan dianggap terlambat.

Terima Kasih!

Do you have any questions? Please use respective class discussion channel on Discord.

Semangat terus menjalani kuilahnya! 🔥🔥🔥